LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Analisis dan Penanganan Deadlock

Kode dan judul tugas	K-3 Analisis dan Penanganan Deadlock
Kelas / tema	Kelas B — Perangkat Mobile dan Aplikasi Android
Nama kelompok	Kelompok 6
Nama host VM	srv-mobile-250660121054
Minggu / tanggal kumpul	Minggu ke-8 / 6 November 2026
Kepingan puzzle	Kepingan 7 → Level 5 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Anggota kelompok

No	Nama	NIM	Peran
1	Nazwa Salsabilla R	250660121077	Leader
2	Satrio Pratama	250660121095	Anggota 1
3	Rizky Imam Andieka	250660121054	Anggota 2

Program Studi S1 Informatika

Fakultas Teknologi Informasi · Universitas Sebelas April

2026

Pernyataan Keaslian

Kami yang bertanda tangan di bawah ini menyatakan bahwa laporan ini merupakan hasil kerja kelompok kami sendiri. Seluruh sumber yang dikutip telah dicantumkan, penggunaan AI generatif telah diungkapkan pada Lampiran, dan setiap anggota berkontribusi sesuai lembar kontribusi.

Nama / NIM	Tanda tangan	Tanggal
Nazwa Salsabilla R - 250660121077		6 November 2026
Satrio Pratama - 250660121095		6 November 2026
Rizky Imam Andieka - 250660121054		6 November 2026

Daftar Isi

Pernyataan Keaslian 2
1. Pendahuluan 4
1.1 Latar belakang kasus 4
1.2 Tujuan tugas 4
1.3 Ruang lingkup 4
2. Pembagian Peran dan Rencana Kerja 5
3. Dasar Teori Singkat 5
4. Lingkungan Praktik 5
5. Langkah Pengerjaan 6
5.1 Langkah 1: Menyiapkan lingkungan dan program 6
5.2 Langkah 2: Merumuskan skenario dan matriks data 6
5.3 Langkah 3: Menggambarkan dan mengidentifikasi empat kondisi 7
5.4 Langkah 4: Menjalankan algoritma Banker 8
6. Hasil dan Analisis 9
6.1 Hasil 9
6.2 Pengujian 11
6.3 Analisis 11
6.4 Rekomendasi 12
7. Kesimpulan 13
8. Kepingan Puzzle 13
Daftar Pustaka 14
Lampiran A. Kode Program dan Log 15
banker.py 15
Keluaran banker.py 17
Lampiran B. Lembar Kontribusi Anggota 19
Lampiran C. Log Kerja Kelompok 19
Lampiran D. Pernyataan Penggunaan AI Generatif 19
Lampiran E. Daftar Periksa Sebelum Mengumpulkan 19
Swa-nilai berdasarkan kriteria lembar tugas 20

1. Pendahuluan

1.1 Latar belakang kasus

Ponsel Android menjalankan banyak aplikasi pada saat yang sama, sedangkan perangkat keras yang diperebutkan, yaitu kamera, GPS, mikrofon, dan penyimpanan, jumlahnya terbatas dan pada praktiknya hanya dapat dipegang satu aplikasi dalam satu waktu. Ketika beberapa aplikasi, misalnya aplikasi media sosial yang merekam video, dashcam navigasi, pencatat lapangan, dan panggilan video, meminta kombinasi sumber daya tersebut secara bersamaan, ada risiko deadlock: setiap aplikasi memegang sebagian sumber daya sambil menunggu sumber daya lain yang dipegang aplikasi lain, sehingga tidak ada satu pun yang dapat melanjutkan.

Masalah ini penting untuk perangkat seluler karena deadlock tampak bagi pengguna sebagai rekaman yang berhenti, peta yang membeku, atau aplikasi yang tidak merespons, dan sulit dipulihkan tanpa menutup paksa aplikasi. Sistem operasi dapat menanganinya dengan mencegah, menghindari, atau mendeteksi deadlock (Silberschatz dkk., 2018, Tanenbaum & Bos, 2023). Laporan ini memodelkan kasus tersebut, menganalisis kapan deadlock terjadi, menentukan apakah suatu keadaan aman dengan algoritma Banker, dan merancang strategi pencegahan yang realistis.

1.2 Tujuan tugas

Capaian pada lembar tugas: Sub-CPMK-7, CPMK061, CPMK063, CPMK081, CPMK102, dan CPMK112 (rumusan lengkap setiap CPMK mengikuti RPS mata kuliah). Dengan kalimat kelompok, tujuan tugas ini adalah sebagai berikut.

- Menggambar resource-allocation graph untuk kasus aplikasi Android yang memakai kamera, GPS, mikrofon, dan penyimpanan, serta mengidentifikasi empat kondisi deadlock pada kasus tersebut.
- Menerapkan algoritma Banker (algoritma keamanan dan algoritma permintaan sumber daya) untuk menentukan apakah keadaan sistem berada pada safe state.
- Merancang strategi pencegahan deadlock yang realistis untuk kasus tersebut, beserta alasan pemilihannya.

1.3 Ruang lingkup

Sistem yang dianalisis adalah model sederhana pengelola sumber daya pada perangkat Android dengan empat aplikasi (P0 Kamera Sosmed, P1 Dashcam Navigasi, P2 Catatan Lapangan, P3 Panggilan Video) dan empat jenis sumber daya (Kamera 3 instance, GPS 2 instance, Mikrofon 2 instance, Penyimpanan 4 instance). Satu instance dipahami sebagai satu slot akses eksklusif yang dapat dipegang satu aplikasi. Data yang dipakai adalah data ilustratif buatan kelompok, bukan matriks yang dibagikan dosen, sehingga angka dapat diganti dengan matriks kelas dengan langkah yang sama. Tugas ini berupa analisis dan simulasi pada VM, pengukuran pada ponsel atau emulator Android, serta implementasi di dalam kernel Android, tidak dikerjakan.

2. Pembagian Peran dan Rencana Kerja

Anggota	Peran	Tanggung jawab	Tenggat
Nazwa Salsabilla R	Perancang strategi dan penyusun laporan	Mengkoordinasi kelompok, menyusun seluruh bagian laporan	6 November 2026
Satrio Pratama	Menganalisis graph	Resource-allocation graph, identifikasi empat kondisi deadlock	6 November 2026
Rizky Imam Andieka	Pelaksana algoritma	Perhitungan algoritma Banker, program banker.py, pengujian	6 November 2026

3. Dasar Teori Singkat

Deadlock adalah keadaan ketika sekumpulan proses saling menunggu sumber daya yang dipegang anggota lain dalam himpunan yang sama, sehingga tidak ada yang dapat maju. Deadlock hanya mungkin terjadi bila empat kondisi berlaku bersamaan: mutual exclusion (sumber daya hanya dipakai satu proses), hold and wait (proses memegang sumber daya sambil menunggu yang lain), no preemption (sumber daya tidak dapat diambil paksa), dan circular wait (ada rantai tunggu melingkar) (Silberschatz dkk., 2018).

Resource-allocation graph (RAG) menggambarkan proses sebagai lingkaran dan jenis sumber daya sebagai kotak yang berisi titik sebanyak instance-nya. Panah dari sumber daya ke proses berarti alokasi, sedangkan panah dari proses ke sumber daya berarti permintaan yang belum terpenuhi. Bila setiap sumber daya hanya punya satu instance, siklus pada graph berarti deadlock; bila sumber daya punya banyak instance, siklus hanya syarat perlu sehingga perlu pemeriksaan lanjutan (Silberschatz dkk., 2018).

Algoritma Banker dipakai untuk penghindaran deadlock. Sistem menyimpan vektor Available serta matriks Allocation, Max, dan Need, dengan Need = Max − Allocation. Algoritma keamanan mencari urutan proses yang semuanya dapat selesai jika ada, keadaan disebut aman (safe state), dan jika tidak, keadaan tidak aman yang berpotensi deadlock. Sebelum mengabulkan permintaan, algoritma permintaan sumber daya mensimulasikan pemberian itu dan hanya mengabulkannya bila keadaan hasilnya tetap aman (Silberschatz dkk., 2018; Tanenbaum & Bos, 2023). Alternatif lain adalah pencegahan, yaitu meniadakan salah satu dari empat kondisi di atas.

4. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	Windows 11 pro, ryzen 5 5500u, ram 16 gb dan Windows 11 pro, ryzen 5 3500U 16GB
Hypervisor	VirtualBox versi

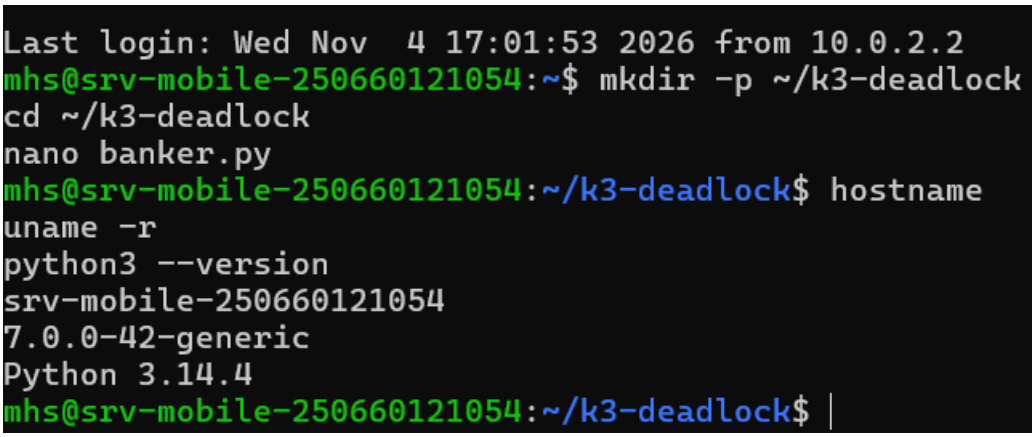
Komponen	Keterangan
VM	srv-mobile-250660121054, Ubuntu 24.04 LTS, 7.0.0-42-generic, 1.60 GB, vCPU : 2 vCPU
Perangkat/emulator lain	Tidak berlaku. Tugas ini berupa analisis dan simulasi pada VM, tidak memakai Android Emulator
Alat dan bahasa	Python 3 (pustaka standar) untuk program banker.py, dan Draw.io
Repositori	https://github.com/NazwaSR0/Tk-3_kelompok6_Sistem-Operasi

5. Langkah Pengerjaan

5.1 Langkah 1: Menyiapkan lingkungan dan program

Tujuan langkah ini adalah memastikan Python 3 tersedia di VM dan menyiapkan folder kerja. Berkas banker.py (Lampiran A) disalin ke folder tersebut.

```
$ hostname
$ uname -r
$ python3 --version
$ mkdir -p ~/k3-deadlock && cd ~/k3-deadlock
# salin banker.py ke folder ini (isi berkas pada Lampiran A)
```



Gambar 5.1 *Persiapan lingkungan pada VM*

5.2 Langkah 2: Merumuskan skenario dan matriks data

Kasus dimodelkan dengan empat aplikasi dan empat jenis sumber daya. Total instance adalah Kamera 3, GPS 2, Mikrofon 2, dan Penyimpanan 4. Matriks Allocation dan Max dirumuskan kelompok sebagai data ilustratif, dengan Max menyatakan jumlah maksimum yang mungkin diminta tiap aplikasi selama berjalan.

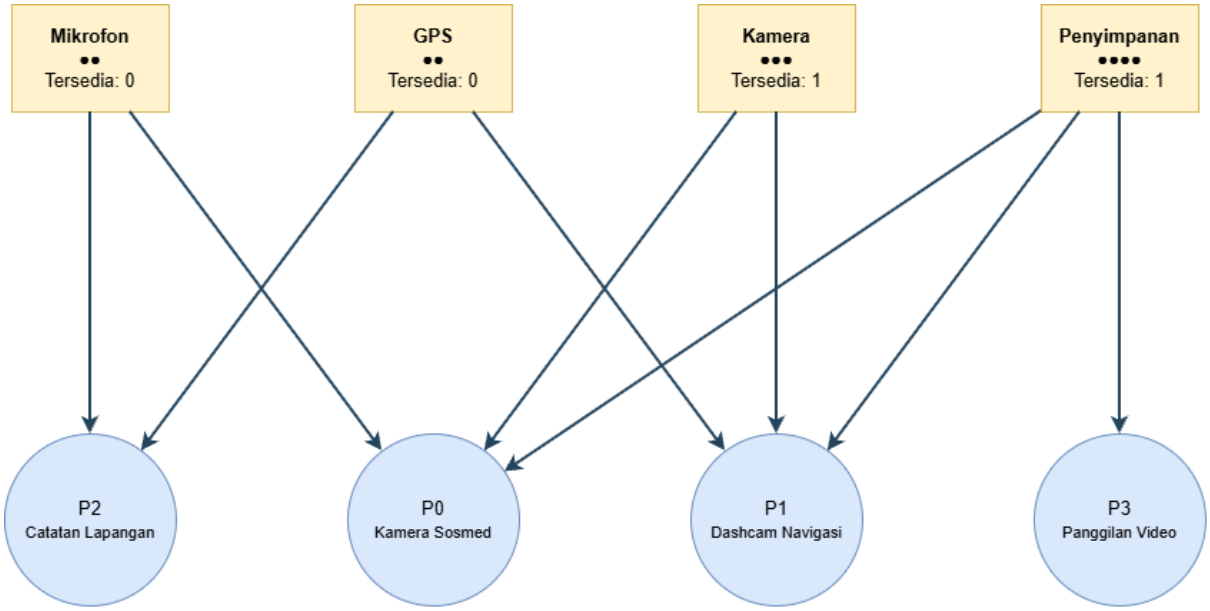
Proses (aplikasi)	Allocation (K, G, M, S)	Max (K, G, M, S)
P0 Kamera Sosmed	(1, 0, 1, 1)	(2, 1, 1, 2)
P1 Dashcam Navigasi	(1, 1, 0, 1)	(1, 2, 1, 2)
P2 Catatan Lapangan	(0, 1, 1, 0)	(1, 1, 1, 1)

Proses (aplikasi)	Allocation (K, G, M, S)	Max (K, G, M, S)
P3 Panggilan Video	(0, 0, 0, 1)	(1, 0, 1, 2)
Total	(3, 2, 2, 4) = total instance	

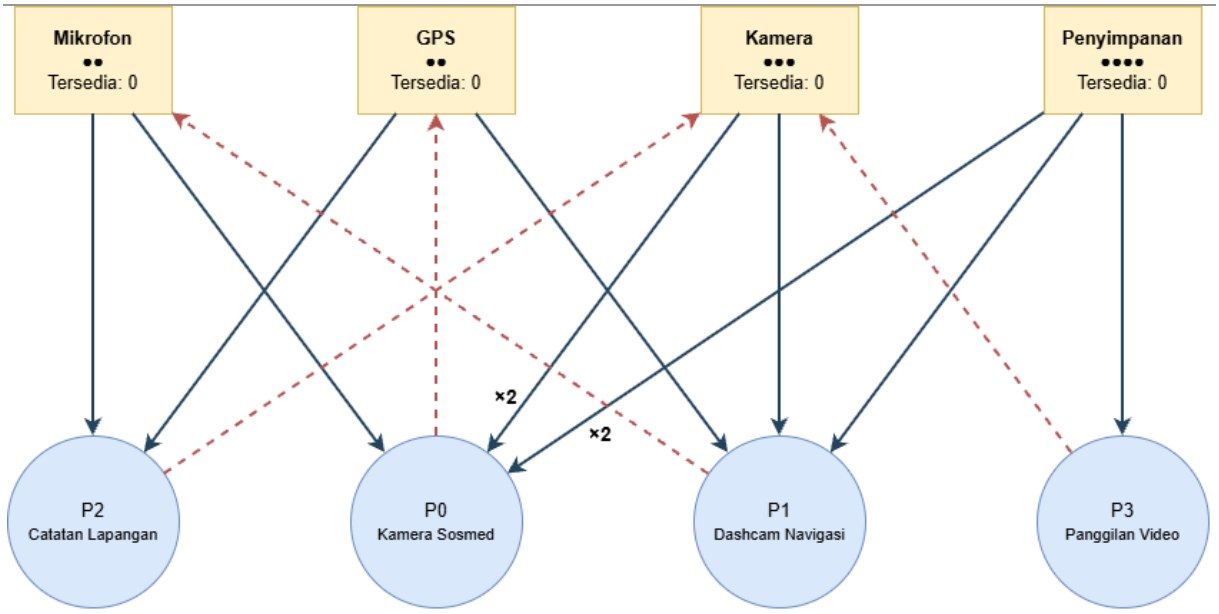
Available dihitung dari total dikurangi jumlah Allocation per kolom: Kamera 3 − 2 = 1, GPS 2 − 2 = 0, Mikrofon 2 − 2 = 0, dan Penyimpanan 4 − 3 = 1, sehingga Available = (1, 0, 0, 1). Need dihitung dengan Need = Max − Allocation.

5.3 Langkah 3: Menggambarkan dan mengidentifikasi empat kondisi

Graph digambar menggunakan web Draw.io. Panah biru dari sumber daya ke proses menyatakan alokasi (label ×2 berarti dua instance), sedangkan panah merah putus-putus dari proses ke sumber daya menyatakan permintaan yang belum terpenuhi. Gambar 5.2 menunjukkan keadaan awal sesuai matriks Allocation, dan Gambar 5.3 menunjukkan keadaan deadlock yang dianalisis pada Bagian 6.



Gambar 5.2 keadaan awal: hanya ada alokasi, tidak ada permintaan tertunda, dan tidak ada siklus



Gambar 5.3 keadaan deadlock: Available (0, 0, 0, 0), P0 menunggu GPS, P1 menunggu Mikrofon, P2 dan P3 menunggu Kamera

5.4 Langkah 4: Menjalankan algoritma Banker

Program banker.py menghitung Need, menjalankan algoritma keamanan, memeriksa empat permintaan sumber daya, mendeteksi deadlock pada keadaan tidak aman, dan menjalankan pengujian otomatis. Keluaran lengkap ada pada Lampiran A.

```
$ python3 banker.py
```

```
mhs@srv-mobile-250660121054:~/k3-deadlock$ python3 banker.py | head -23
Sumber daya : ['Kamera', 'GPS', 'Mikrofon', 'Penyimpanan']
Total       : (3,2,2,4)
Available   : (1,0,0,1)

Proses  Allocation  Max       Need
P0      (1,0,1,1)   (2,1,1,2)  (1,1,0,1)   <- Kamera Sosmed
P1      (1,1,0,1)   (1,2,1,2)  (0,1,1,1)   <- Dashcam Navigasi
P2      (0,1,1,0)   (1,1,1,1)  (1,0,0,1)   <- Catatan Lapangan
P3      (0,0,0,1)   (1,0,1,2)  (1,0,1,1)   <- Panggilan Video

== 1. Algoritma keamanan (safe state) ==
Iterasi 1: P2 Need(1,0,0,1) <= Work(1,0,0,1) -> selesai, Work baru = (1,1,1,1)
Iterasi 2: P0 Need(1,1,0,1) <= Work(1,1,1,1) -> selesai, Work baru = (2,1,2,2)
Iterasi 3: P1 Need(0,1,1,1) <= Work(2,1,2,2) -> selesai, Work baru = (3,2,2,3)
Iterasi 4: P3 Need(1,0,1,1) <= Work(3,2,2,3) -> selesai, Work baru = (3,2,2,4)
Hasil: AMAN | urutan aman: P2 -> P0 -> P1 -> P3

== 2. Permintaan sumber daya ==
P2 meminta (1,0,0,0) -> DIBERIKAN: state tetap aman, urutan aman: P2 -> P0 -> P1 -> P3
P0 meminta (1,0,0,1) -> DITOLAK: state hasil pemberian tidak aman
P1 meminta (0,1,0,0) -> TUNGGU: sumber daya belum tersedia (Request > Available)
P3 meminta (0,1,0,0) -> ERROR: permintaan melebihi Need (melanggar klaim maksimum)
```

Gambar 5.4 *Keluaran algoritma keamanan dan algoritma permintaan sumber daya*

```
mhs@srv-mobile-250660121054:~/k3-deadlock$ python3 banker.py | tail -14
== 3. Deteksi deadlock pada state tidak aman ==
Available (0,0,0,0) | Request P0..P3 = (0,1,0,0) (0,0,1,0) (1,0,0,0) (1,0,0,0)
Proses yang deadlock: P0, P1, P2, P3

== 4. Pengujian otomatis ==
[LULUS] Available awal = (1,0,0,1)
[LULUS] Need P0..P3 sesuai perhitungan manual
[LULUS] State awal aman, urutan P2,P0,P1,P3
[LULUS] P2 minta (1,0,0,0) -> DIBERIKAN
[LULUS] P0 minta (1,0,0,1) -> DITOLAK (tidak aman)
[LULUS] P1 minta (0,1,0,0) -> TUNGGU (GPS habis)
[LULUS] P3 minta (0,1,0,0) -> ERROR (> Need)
[LULUS] Deteksi: P0,P1,P2,P3 deadlock
Ringkasan: 8 dari 8 uji lulus
```

Gambar 5.5 *Keluaran deteksi deadlock dan pengujian otomatis*

6. Hasil dan Analisis

6.1 Hasil

Tabel 6.1 Matriks Need. Need = Max − Allocation, dengan Available awal (1, 0, 0, 1).

Proses	Allocation	Max	Need
P0	(1, 0, 1, 1)	(2, 1, 1, 2)	(1, 1, 0, 1)
P1	(1, 1, 0, 1)	(1, 2, 1, 2)	(0, 1, 1, 1)
P2	(0, 1, 1, 0)	(1, 1, 1, 1)	(1, 0, 0, 1)
P3	(0, 0, 0, 1)	(1, 0, 1, 2)	(1, 0, 1, 1)

Kelompok 6 – K-3 Analisis Deadlock :

Proses	Alloc Kamera	Alloc GPS	Alloc Mikrofon	Alloc Penyimpanan	Max Kamera	Max GPS	Max Mikrofon	Max Penyimpanan	Need Kamera	Need GPS	Need Mikro
P0	1	0	1	1	2	1	1	2	1	1	0
P1	1	1	0	1	1	2	1	2	0	1	1
P2	0	1	1	0	1	1	1	1	1	0	0
P3	0	0	0	1	1	0	1	2	1	0	1
Total instance	3	2	2	4							
Jumlah Allocation	2	2	2	3							
Available	1	0	0	1							

Algoritma keamanan (safe state)

Iterasi / proses	Work sblm Kamera	Work sblm GPS	Work sblm Mikrofon	Work sblm Penyimpanan	Work ssdh Kamera	Work ssdh GPS	Work ssdh Mikrofon	Work ssdh Penyimpanan	Need ≤ Work?
1. P2	1	0	0	1	1	1	1	1	Ya
2. P0	1	1	1	1	2	1	2	2	Ya
3. P1	2	1	2	2	3	2	2	3	Ya
4. P3	3	2	2	3	3	2	2	4	Ya
Hasil	AMAN (safe state). Urutan aman: P2 → P0 → P1 → P3								

Tabel 6.2 Algoritma keamanan. Setiap iterasi memilih proses dengan Need ≤ Work, lalu menambahkan Allocation proses itu ke Work.

Iterasi	Proses	Need	Work sebelum	Work sesudah
1	P2	(1, 0, 0, 1)	(1, 0, 0, 1)	(1, 1, 1, 1)
2	P0	(1, 1, 0, 1)	(1, 1, 1, 1)	(2, 1, 2, 2)
3	P1	(0, 1, 1, 1)	(2, 1, 2, 2)	(3, 2, 2, 3)

Iterasi	Proses	Need	Work sebelum	Work sesudah
4	P3	(1, 0, 1, 1)	(3, 2, 2, 3)	(3, 2, 2, 4)

Semua proses dapat selesai, dan Work akhir (3, 2, 2, 4) sama dengan total instance. Keadaan awal berada pada **safe state** dengan urutan aman P2 → P0 → P1 → P3.

Tabel 6.3 Permintaan sumber daya (Available awal (1, 0, 0, 1)).

Peminta	Request	Hasil	Alasan
P2	(1, 0, 0, 0)	Diberikan	Request ≤ Need dan ≤ Available. Setelah pemberian, Available (0, 0, 0, 1) dan keadaan tetap aman: P2 → P0 → P1 → P3.
P0	(1, 0, 0, 1)	Ditolak	Request ≤ Need dan ≤ Available, tetapi setelah pemberian Available menjadi (0, 0, 0, 0) dan tidak ada proses dengan Need ≤ Work, sehingga keadaan tidak aman.
P1	(0, 1, 0, 0)	Menunggu	Request ≤ Need, tetapi GPS belum tersedia (Available GPS = 0).
P3	(0, 1, 0, 0)	Error	Request melebihi Need (Need GPS P3 = 0), melanggar klaim maksimum.

Tabel 6.4 Keadaan deadlock. Keadaan ini muncul bila permintaan P0 (1, 0, 0, 1) tetap dikabulkan, lalu setiap aplikasi meminta satu unit dari sisa kebutuhannya.

Proses	Allocation	Request	Available	Hasil deteksi
P0	(2, 0, 1, 2)	(0, 1, 0, 0)	(0, 0, 0, 0)	Deadlock
P1	(1, 1, 0, 1)	(0, 0, 1, 0)		Deadlock
P2	(0, 1, 1, 0)	(1, 0, 0, 0)		Deadlock
P3	(0, 0, 0, 1)	(1, 0, 0, 0)		Deadlock

Tabel 6.5 Empat kondisi deadlock pada keadaan Gambar 5.3.

Kondisi	Bukti pada kasus
Mutual exclusion	Satu instance kamera, GPS, mikrofon, atau slot penyimpanan hanya dapat dipakai satu aplikasi pada satu waktu.
Hold and wait	P0 memegang Kamera ×2, Mikrofon, dan Penyimpanan ×2 sambil meminta GPS. P1 memegang Kamera, GPS, dan Penyimpanan sambil meminta Mikrofon.
No preemption	Dalam skenario ini sumber daya hanya dilepas secara sukarela oleh pemegangnya. Tidak ada mekanisme yang mengambil paksa kamera, GPS, atau mikrofon dari aplikasi yang sedang memakainya.

Kondisi	Bukti pada kasus
Circular wait	P0 → GPS → P1 → Mikrofon → P0 membentuk siklus. Siklus lain: P0 → GPS → P2 → Kamera → P0.

6.2 Pengujian

Pengujian dijalankan otomatis oleh banker.py. Hasil aktual diambil dari keluaran program (Lampiran A) dan dibandingkan dengan perhitungan manual pada Bagian 6.1.

No	Skenario uji	Hasil diharapkan	Hasil aktual	Status
1	Hitung Available awal	(1, 0, 0, 1)	(1, 0, 0, 1)	Lulus
2	Hitung matriks Need	Sesuai Tabel 6.1	Sesuai Tabel 6.1	Lulus
3	Algoritma keamanan pada keadaan awal	Aman, P2, P0, P1, P3	Aman, P2, P0, P1, P3	Lulus
4	P2 meminta (1, 0, 0, 0)	Diberikan	Diberikan	Lulus
5	P0 meminta (1, 0, 0, 1)	Ditolak (tidak aman)	Ditolak	Lulus
6	P1 meminta (0, 1, 0, 0)	Menunggu (GPS habis)	Menunggu	Lulus
7	P3 meminta (0, 1, 0, 0)	Error (melebihi Need)	Error	Lulus
8	Deteksi deadlock pada Tabel 6.4	P0, P1, P2, P3 deadlock	P0, P1, P2, P3 deadlock	Lulus

6.3 Analisis

Poin 1: resource-allocation graph dan empat kondisi deadlock. Pada keadaan awal (Gambar 5.2) graph hanya berisi panah alokasi tanpa panah permintaan, sehingga tidak ada siklus dan tidak ada deadlock, sistem hanya sedang sibuk. Deadlock baru muncul pada Gambar 5.3 ketika semua instance habis dan setiap aplikasi meminta sumber daya yang dipegang aplikasi lain. Karena sumber daya di sini punya banyak instance, adanya siklus saja belum cukup untuk menyimpulkan deadlock. Kesimpulan deadlock diperoleh dari algoritma deteksi: dengan Work (0, 0, 0, 0), tidak ada aplikasi yang Request-nya terpenuhi, sehingga keempatnya tidak pernah dapat selesai. Keempat kondisi deadlock pada Tabel 6.5 terpenuhi sekaligus, dan itulah yang membuat keadaan ini macet permanen.

Poin 2: algoritma Banker dan safe state. Meski Available awal hanya (1, 0, 0, 1), keadaan awal aman karena P2 membutuhkan tepat (1, 0, 0, 1). Setelah P2 selesai dan mengembalikan GPS serta Mikrofon, Work bertambah sehingga P0, P1, dan P3 dapat menyusul secara berurutan. Contoh yang paling penting adalah permintaan P0 (1, 0, 0, 1). Permintaan itu valid dan tersedia saat itu juga, tetapi pemberiannya menghabiskan seluruh Available dan membuat tidak ada aplikasi yang bisa selesai. Keadaan hasilnya persis keadaan deadlock pada Gambar 5.3. Jadi algoritma Banker mencegah deadlock dengan menolak permintaan yang tampak aman bagi peminta tetapi tidak aman bagi sistem. Permintaan P2 diberikan karena keadaan hasilnya

tetap aman, permintaan P1 menunggu karena GPS belum tersedia, dan permintaan P3 ditolak sebagai kesalahan karena melampaui klaim maksimumnya.

Poin 3: strategi pencegahan yang realistis. Pencegahan bekerja dengan meniadakan satu dari empat kondisi. Mutual exclusion tidak realistis ditiadakan karena kamera dan mikrofon memang tidak dapat dibagi sembarangan, sehingga strategi diarahkan pada tiga kondisi lain, seperti pada Tabel 6.6.

Tabel 6.6 Strategi pencegahan.

Kondisi yang ditiadakan	Strategi	Pada kasus ini	Kelebihan dan kekurangan
Circular wait	Urutan global sumber daya: Kamera (1) < GPS (2) < Mikrofon (3) < Penyimpanan (4). Aplikasi hanya boleh meminta sumber daya bernomor lebih besar dari yang dipegangnya.	Keempat permintaan pada Gambar 5.3 melanggar urutan (misalnya P0 memegang Mikrofon lalu meminta GPS) sehingga ditolak sejak awal dan siklus tidak dapat terbentuk.	Murah dan deterministik, cocok untuk pengelola sumber daya sistem. Aplikasi harus mengikuti urutan, dan kadang meminta sumber daya lebih awal dari yang dibutuhkan.
Hold and wait	Permintaan sekaligus (all-or-nothing): aplikasi meminta seluruh sumber daya yang dibutuhkan, dan bila ada yang belum tersedia ia tidak memegang apa pun.	P0 yang butuh GPS tidak akan memegang Kamera dan Penyimpanan sambil menunggu.	Menghilangkan penahanan sambil menunggu. Utilisasi turun dan aplikasi besar berisiko starvation.
No preemption	Timeout dan pengambilan paksa: sumber daya yang ditahan melewati batas waktu, atau diminta aplikasi berprioritas lebih tinggi (misalnya yang sedang di layar), dilepas oleh sistem.	Kamera dan mikrofon dapat ditarik dari aplikasi yang menunggu terlalu lama sehingga rantai tunggu putus.	Cocok untuk sumber daya yang mudah dilanjutkan seperti kamera dan mikrofon. Kurang cocok untuk penyimpanan yang sedang menulis, karena perlu rollback.

6.4 Rekomendasi

Kelompok merekomendasikan kombinasi urutan global sumber daya dan permintaan sekaligus untuk kelompok sumber daya yang selalu dipakai bersama, ditambah timeout dengan prioritas

aplikasi di layar untuk kamera dan mikrofon. Kombinasi ini dipilih karena biayanya kecil dan tidak mensyaratkan aplikasi menyatakan Max di muka. Algoritma Banker sebaiknya dipakai terbatas pada sumber daya yang kebutuhan maksimumnya dapat dideklarasikan, misalnya kuota penyimpanan, karena algoritma keamanan membutuhkan komputasi sekitar $m \times n^2$ untuk n proses dan m jenis sumber daya, dan klaim Max sulit dipastikan untuk aplikasi pihak ketiga (Silberschatz dkk., 2018). Keterbatasan: model memakai data ilustratif dan menganggap satu instance sebagai satu slot eksklusif, sehingga hasilnya perlu diuji ulang dengan matriks kelas dan sistem sebenarnya.

7. Kesimpulan

Pada keadaan awal, sistem berada pada safe state dengan urutan aman P2 → P0 → P1 → P3, dan graph awal tidak mengandung siklus. Deadlock terjadi bila permintaan P0 (1, 0, 0, 1) dikabulkan lalu setiap aplikasi meminta sumber daya berikutnya, karena keempat kondisi deadlock terpenuhi sekaligus dan semua aplikasi saling menunggu. Algoritma Banker menolak permintaan P0 tersebut sehingga keadaan deadlock dapat dihindari. Strategi pencegahan yang realistis untuk kasus ini adalah urutan global sumber daya, permintaan sekaligus, serta timeout dan prioritas, yang masing-masing meniadakan circular wait, hold and wait, dan no preemption.

8. Kepingan Puzzle

Nomor kepingan	#7
Isi kepingan	Resource-allocation graph, perhitungan algoritma Banker (Need, safe state, permintaan sumber daya), tabel empat kondisi deadlock, rancangan strategi pencegahan, dan program banker.py
Lokasi di portofolio	https://drive.google.com/drive/folders/1CO-Zv-dtNRRKFciASna8vlSB0CrVwtsA
Dipakai untuk	Level 5 Puzzle UTS kepingan #7, sebagai bukti analisis dan penanganan deadlock pada kasus aplikasi Android

Daftar Pustaka

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2023). *Modern operating systems* (5th ed.). Pearson.

Lampiran A. Kode Program dan Log

banker.py

```
#!/usr/bin/env python3
"""K-3 Analisis dan Penanganan Deadlock - Algoritma Banker (Kelas B, Kelompok 6).

Simulasi pengelolaan sumber daya perangkat (kamera, GPS, mikrofon, penyimpanan)
oleh beberapa aplikasi Android. Hanya memakai pustaka standar Python 3.
Jalankan: python3 banker.py
"""
RES = ["Kamera", "GPS", "Mikrofon", "Penyimpanan"]
PROC = ["P0", "P1", "P2", "P3"]
NAMA = {"P0": "Kamera Sosmed", "P1": "Dashcam Navigasi",
        "P2": "Catatan Lapangan", "P3": "Panggilan Video"}

TOTAL = [3, 2, 2, 4]
ALLOC = [[1, 0, 1, 1], [1, 1, 0, 1], [0, 1, 1, 0], [0, 0, 0, 1]]
MAX_ = [[2, 1, 1, 2], [1, 2, 1, 2], [1, 1, 1, 1], [1, 0, 1, 2]]


def available(total, alloc):
    return [total[j] - sum(a[j] for a in alloc) for j in range(len(total))]


def need(max_, alloc):
    return [[m - a for m, a in zip(mr, ar)] for mr, ar in zip(max_, alloc)]


def leq(a, b):
    return all(x <= y for x, y in zip(a, b))


def is_safe(avail, alloc, need_, trace=None):
    """Algoritma keamanan. Mengembalikan (aman, urutan_aman)."""
    n = len(alloc)
    work, finish, seq = list(avail), [False] * n, []
    while len(seq) < n:
        for i in range(n):
            if not finish[i] and leq(need_[i], work):
                if trace is not None:
                    trace.append((PROC[i], list(need_[i]), list(work)))
                work = [w + a for w, a in zip(work, alloc[i])]
                finish[i] = True
                seq.append(PROC[i])
                if trace is not None:
                    trace[-1] += (list(work),)
                break
        else:
            return False, seq
    return True, seq
```

```
def request(i, req, avail, alloc, need_):
    """Algoritma permintaan sumber daya. Mengembalikan (status, keterangan)."""
    if not leq(req, need_[i]):
        return "ERROR", "permintaan melebihi Need (melanggar klaim maksimum)"
    if not leq(req, avail):
        return "TUNGGU", "sumber daya belum tersedia (Request > Available)"
    av2 = [a - r for a, r in zip(avail, req)]
    al2 = [r[:] for r in alloc]
    nd2 = [r[:] for r in need_]
    al2[i] = [a + r for a, r in zip(al2[i], req)]
    nd2[i] = [a - r for a, r in zip(nd2[i], req)]
    ok, seq = is_safe(av2, al2, nd2)
    if ok:
        return "DIBERIKAN", "state tetap aman, urutan aman: " + " -> ".join(seq)
    return "DITOLAK", "state hasil pemberian tidak aman"


def detect_deadlock(avail, alloc, req_matrix):
    """Deteksi deadlock (matriks Request). Mengembalikan daftar proses deadlock."""
    n = len(alloc)
    work = list(avail)
    finish = [all(x == 0 for x in alloc[i]) for i in range(n)]
    changed = True
    while changed:
        changed = False
        for i in range(n):
            if not finish[i] and leq(req_matrix[i], work):
                work = [w + a for w, a in zip(work, alloc[i])]
                finish[i] = True
                changed = True
    return [PROC[i] for i in range(n) if not finish[i]]


def fmt(v):
    return "(" + ",".join(str(x) for x in v) + ")"


def main():
    av = available(TOTAL, ALLOC)
    nd = need(MAX_, ALLOC)
    print("Sumber daya :", RES)
    print("Total       :", fmt(TOTAL))
    print("Available   :", fmt(av))
    print("\nProses  Allocation  Max       Need")
    for i, p in enumerate(PROC):
        print(f"{p:6}  {fmt(ALLOC[i]):10}  {fmt(MAX_[i]):8}  {fmt(nd[i])}   <-
{NAMA[p]}")
```

```
    print("\n== 1. Algoritma keamanan (safe state) ==")
    trace = []
    ok, seq = is_safe(av, ALLOC, nd, trace)
    for k, (p, ne, w, w2) in enumerate(trace, 1):
        print(f"Iterasi {k}: {p} Need{fmt(ne)} <= Work{fmt(w)} -> selesai, Work baru =
{fmt(w2)}")
    print("Hasil:", "AMAN" if ok else "TIDAK AMAN", "| urutan aman:", " -> ".join(seq))

    print("\n== 2. Permintaan sumber daya ==")
    for i, req in [(2, [1, 0, 0, 0]), (0, [1, 0, 0, 1]), (1, [0, 1, 0, 0]), (3, [0, 1, 0, 0])]:
        st, ket = request(i, req, av, ALLOC, nd)
        print(f"{PROC[i]} meminta {fmt(req)} -> {st}: {ket}")

    print("\n== 3. Deteksi deadlock pada state tidak aman ==")
    al2 = [r[:] for r in ALLOC]
    al2[0] = [a + r for a, r in zip(al2[0], [1, 0, 0, 1])]
    av2 = [a - r for a, r in zip(av, [1, 0, 0, 1])]
    reqm = [[0, 1, 0, 0], [0, 0, 1, 0], [1, 0, 0, 0], [1, 0, 0, 0]]
    print("Available", fmt(av2), "| Request P0..P3 =", " ".join(fmt(r) for r in reqm))
    dl = detect_deadlock(av2, al2, reqm)
    print("Proses yang deadlock:", ", ".join(dl) if dl else "tidak ada")

    print("\n== 4. Pengujian otomatis ==")
    hasil = []
    def uji(nama, cond):
        hasil.append(cond)
        print(f"[{'LULUS' if cond else 'GAGAL'}] {nama}")
    uji("Available awal = (1,0,0,1)", av == [1, 0, 0, 1])
    uji("Need P0..P3 sesuai perhitungan manual", nd ==
[[1,1,0,1],[0,1,1,1],[1,0,0,1],[1,0,1,1]])
    uji("State awal aman, urutan P2,P0,P1,P3", ok and seq == ["P2","P0","P1","P3"])
    uji("P2 minta (1,0,0,0) -> DIBERIKAN",
request(2,[1,0,0,0],av,ALLOC,nd)[0]=="DIBERIKAN")
    uji("P0 minta (1,0,0,1) -> DITOLAK (tidak aman)",
request(0,[1,0,0,1],av,ALLOC,nd)[0]=="DITOLAK")
    uji("P1 minta (0,1,0,0) -> TUNGGU (GPS habis)",
request(1,[0,1,0,0],av,ALLOC,nd)[0]=="TUNGGU")
    uji("P3 minta (0,1,0,0) -> ERROR (> Need)",
request(3,[0,1,0,0],av,ALLOC,nd)[0]=="ERROR")
    uji("Deteksi: P0,P1,P2,P3 deadlock", dl == PROC)
    print("Ringkasan:", sum(hasil), "dari", len(hasil), "uji lulus")


if __name__ == "__main__":
    main()
```

Keluaran banker.py

```
Sumber daya : ['Kamera', 'GPS', 'Mikrofon', 'Penyimpanan']
Total       : (3,2,2,4)
```

```
Available   : (1,0,0,1)

Proses  Allocation  Max       Need
P0      (1,0,1,1)   (2,1,1,2)  (1,1,0,1)   <- Kamera Sosmed
P1      (1,1,0,1)   (1,2,1,2)  (0,1,1,1)   <- Dashcam Navigasi
P2      (0,1,1,0)   (1,1,1,1)  (1,0,0,1)   <- Catatan Lapangan
P3      (0,0,0,1)   (1,0,1,2)  (1,0,1,1)   <- Panggilan Video

== 1. Algoritma keamanan (safe state) ==
Iterasi 1: P2 Need(1,0,0,1) <= Work(1,0,0,1) -> selesai, Work baru = (1,1,1,1)
Iterasi 2: P0 Need(1,1,0,1) <= Work(1,1,1,1) -> selesai, Work baru = (2,1,2,2)
Iterasi 3: P1 Need(0,1,1,1) <= Work(2,1,2,2) -> selesai, Work baru = (3,2,2,3)
Iterasi 4: P3 Need(1,0,1,1) <= Work(3,2,2,3) -> selesai, Work baru = (3,2,2,4)
Hasil: AMAN | urutan aman: P2 -> P0 -> P1 -> P3

== 2. Permintaan sumber daya ==
P2 meminta (1,0,0,0) -> DIBERIKAN: state tetap aman, urutan aman: P2 -> P0 -> P1 -> P3
P0 meminta (1,0,0,1) -> DITOLAK: state hasil pemberian tidak aman
P1 meminta (0,1,0,0) -> TUNGGU: sumber daya belum tersedia (Request > Available)
P3 meminta (0,1,0,0) -> ERROR: permintaan melebihi Need (melanggar klaim maksimum)

== 3. Deteksi deadlock pada state tidak aman ==
Available (0,0,0,0) | Request P0..P3 = (0,1,0,0) (0,0,1,0) (1,0,0,0) (1,0,0,0)
Proses yang deadlock: P0, P1, P2, P3

== 4. Pengujian otomatis ==
[LULUS] Available awal = (1,0,0,1)
[LULUS] Need P0..P3 sesuai perhitungan manual
[LULUS] State awal aman, urutan P2,P0,P1,P3
[LULUS] P2 minta (1,0,0,0) -> DIBERIKAN
[LULUS] P0 minta (1,0,0,1) -> DITOLAK (tidak aman)
[LULUS] P1 minta (0,1,0,0) -> TUNGGU (GPS habis)
[LULUS] P3 minta (0,1,0,0) -> ERROR (> Need)
[LULUS] Deteksi: P0,P1,P2,P3 deadlock
Ringkasan: 8 dari 8 uji lulus
```

Lampiran B. Lembar Kontribusi Anggota

Persentase kontribusi berjumlah 100% dan disepakati seluruh anggota.

No	Nama	NIM	Peran dan kontribusi	%	Paraf
1	Nazwa Salsabilla R	250660121077	Mengkoordinasikan kelompok, menyusun seluruh isi laporan	36%	
2	Satrio Pratama	250660121095	Membuat graph pada web draw.io	32%	
3	Rizky Imam Andieka	250660121054	Menjalankan program pada vm	32%	
	Total			100	

Lampiran C. Log Kerja Kelompok

Tanggal	Kegiatan	Anggota hadir	Hasil
4 November 2026	Pembahasan pembagian tugas	Semua anggota	Kasaran mengenai proses tugas
5 November 2026	Mengerjakan tugas masing masing dan pembuatan laporan	Semua anggota	Hasil program, graph, laporan

Lampiran D. Pernyataan Penggunaan AI Generatif

☐ Kelompok kami tidak menggunakan AI generatif dalam tugas ini.

☑ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude	Membantu menyusun kerangka laporan, data ilustratif, program banker.py, dan membantu menjelaskan soal yg kurang dipahami	Rekomendasi, lampiran A, bagian pendahuluan	Pengecekan program yg dijalankan di VM

Lampiran E. Daftar Periksa Sebelum Mengumpulkan

☑ Semua teks penanda […] berwarna kuning sudah diisi atau dihapus.

☑ Identitas, kode tugas, dan nama host VM pada sampul sudah benar.

☑ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☑ Setiap tangkapan layar menampilkan prompt/nama host VM dan memiliki nomor serta keterangan.

☑ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.

☑ Lembar kontribusi sudah diisi dan disetujui semua anggota (total 100%).

☑ Kode dan konfigurasi sudah diunggah ke repositori kelompok.

☑ Pernyataan penggunaan AI generatif sudah diisi.

☑ Daftar isi sudah diperbarui (klik kanan → Update Field).

☑ Berkas disimpan sebagai PDF dengan nama B_K-3_Kelompok6.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana?
Hasil/keberhasilan implementasi	35%	Ya: Bagian 5, 6.1, 6.2, dan Lampiran A (setelah tangkapan layar ditempel)
Kedalaman analisis	30%	Ya: Bagian 6.3 dan 6.4
Dokumentasi	20%	Ya: seluruh laporan, Lampiran A sampai E, dan daftar pustaka
Kerja sama dan pembagian peran	15%	Ya: Bagian 2, Lampiran B, dan Lampiran C (setelah diisi)